MetaSpace Membrane

Deterministic Containment for Machine-Generated Software

An undecidable question turned into a decidable one, and the answer enforced deny-by-default.

Abstract

Machine-generated software cannot be verified for *intent*: by Rice's theorem, whether an arbitrary program conforms to its author's intent is undecidable. MetaSpace Membrane replaces verification of intent with *containment of effect*. A program's effects are constrained to a boundary declared in a single, deliberately Turing-incomplete `.bio` constitution, and any effect outside the boundary is deterministically refused. Containment reduces to *set membership*, which is decidable, so the runtime decision is deterministic and reproducible. We describe three composable membranes (capability, value-invariant, epistemic), a provenance chain (synthesis → human ratification → production gate), and an evidence ledger of twelve reproducible proofs run by a single command. We are explicit about what is *hard* (unbypassable under the WebAssembly capability model) and what is *soft* (semantic faithfulness, non-deterministic and non-enforcing).

1. Problem — intent is undecidable

The correctness question — *does the program do what its author intended?* — is undecidable: Rice's theorem states that every non-trivial semantic property of programs is undecidable. For machine-generated code the gap is acute: the generator's intent is not mechanically checkable, and a plausible-looking program may take effects its author never sanctioned — deleting files, exfiltrating data, or acting on invented facts. Verifying *correctness* therefore does not scale to this setting.

2. Approach — containment, not correctness

We substitute a decidable question. Instead of *"is the program right?"* we ask *"is the program's effect inside the declared boundary?"* — a set-membership test, which is decidable and deterministic. The boundary is a `.bio` constitution, deliberately Turing-incomplete, so the policy is finite and analyzable. A "bug" is redefined as *a deviation from the constitution*: machine-measurable and enforceable. [ARCH]

3. Architecture — one pattern, three membranes

All three membranes are the same reference-monitor pattern at a chokepoint, **deny-by-default**, with the decision made by the constitution.

- **Capability membrane** — mediates effects on the world (filesystem, network, subprocess, …).
- **Value-invariant membrane** — guards declared safety bounds.
- **Epistemic membrane** — bounds factual assertion and knowledge-based actuation.

Guarantee ladder. A membrane is only as hard as its chokepoint is unbypassable: heuristic (advisory) < language-level (bypassable via `ctypes`/syscalls) < harness-level (hard for the agent, since it sits outside it) < WebAssembly (unbypassable: a guest has no ambient authority). [ARCH]

Composition. The epistemic membrane alone is only advisory — an app may simply not call it. It becomes hard *behind* the capability membrane, which guarantees the sole path to the world is a mediated gate. The three compose in defence-in-depth. [ARCH]

4. Provenance chain — synthesis, ratification, gate

- **Synthesis.** Static analysis of code drafts a constitution, marked SYNTHESIZED. This is a name-based heuristic, not a guarantee. [ARCH]
- **Ratification.** A human reviews and stamps it RATIFIED. The stamp is *content-bound*: it carries a fingerprint of the enforced policy, so a later edit that widens a scope or adds a capability reads as TAMPERED. [MEASURED]
- **Gate.** In production, only a RATIFIED constitution runs; a SYNTHESIZED or TAMPERED one is refused, fail-closed. A policy is enforceable only after human ratification, and cannot be broadened afterwards without detection. [MEASURED]

5. Evidence — reproducible ledger (`python run_proofs.py`)

Every row below is a real run; all seventeen reproduce together with one command, result **17/17** (the exact command is in the closing section).

Claim	Status	Evidence (observed output)
Capability membrane mediates every capability kind (custom host imports)	[MEASURED]	`run_wasm_demo`: 4 ALLOW / 5 DENY across FS write+read, NETWORK, ENV, SUBPROCESS (each scope-enforced); the forbidden write is physically absent
Unbypassability under WebAssembly	[MEASURED] + [ARCH]	an ungranted gate (WASI `path_open`) fails to link: *"unknown import … has not been defined"*
A real compiled program is contained by WASI capabilities	[MEASURED]	Rust → `wasm32-wasip1`: reads a read-only input (READ), writes a writable output (WROTE), is refused when writing into the read-only input (`ENOTCAPABLE`) or outside all preopens
Code → constitution → enforcement (closed loop)	[MEASURED]	the synthesized policy bounds the app; an *undeclared* subprocess is denied by default, and an out-of-allowlist network host is denied
Ratification is content-bound	[MEASURED]	SYNTHESIZED → RATIFIED → TAMPERED when the policy is widened after ratifying
Production gate admits only RATIFIED	[MEASURED]	SYNTHESIZED and TAMPERED are REFUSED; RATIFIED RUNS and then enforces
Ratification cognitive brake (provisional capabilities)	[MEASURED]	a dry-run-learned capability cannot be ratified without a written justification; a smuggled, unjustified one is refused (`--yes` cannot bypass it)
Epistemic hard tier (referential integrity, schema/domain, provenance-locked actuation, citation)	[MEASURED]	`run_knowledge_demo`: 2 ALLOW / 5 DENY (invented entity, out-of-domain enum, dangling reference, missing citation, ungrounded actuation)
Epistemic soft tier is an *advisory flag*, not a membrane	[MEASURED] + [ARCH]	a faithful claim is flagged SUPPORTED, an invented figure UNSUPPORTED; it qualifies, never contains
Agent membrane mediates the coding agent's own tool calls	[MEASURED]	PreToolUse hook, 17/17 cases: project-only writes, network host allowlist, structural shell block, fail-closed on unreadable input
Structural shell policy (allowlist, not substring)	[MEASURED]	a lexer resolves real program names; catches obfuscations the substring denylist misses (quoting, `${IFS}`, chaining) and fails closed on garbage
Dry-run "learning mode" closes the false-positive gap	[MEASURED]	a static-only constitution DENIES a program's real runtime write (false positive); the dry-run-augmented one ALLOWS it
Threat-model matrix (honest layer coverage)	[MEASURED]	each layer's real check per attack vector; a schema-valid but fabricated statement PASSES the hard layers and is only FLAGGED by the soft tier
Real program does real work under containment	[MEASURED]	a real Rust log analyzer (dir scan, parse, aggregate) computes the correct stats for known input; its out-of-grant read is refused by WASI
Synthesizer runs on real code (dogfood)	[MEASURED]	run over this repo's own ∼36 files, it produces a real, non-trivial constitution matching the code's actual effects
Property-based fuzz (adversarial coverage)	[MEASURED]	5000 random constitutions × random effects: deny-by-default never violated; the guard agrees with an independent scope oracle
Self-falsification (anti-slop)	[MEASURED]	mutating the `.bio` flips the decision; sabotaging `guard.check` fails a proof; wasmtime itself refuses an ungranted

6. Scope, trust boundary & honest limits

- The *hard* guarantee holds under the WebAssembly capability model. A language-level guard is bypassable (`ctypes`, direct syscalls) and is therefore *not* shipped as a product.
- Constitution synthesis is a static, name-based heuristic; it can under-declare under heavy dynamism. A **dry-run learning mode** observes concrete runtime effects and augments the constitution *before* ratification, so an incomplete policy does not false-positive-block legitimate behaviour. The runtime membrane — not the synthesis — remains the guarantee.
- The epistemic *soft* tier (semantic faithfulness) is non-deterministic and does not enforce; it is an **advisory flag**, not a membrane — it *qualifies* (flags), never *contains* (blocks). A reproducible threat-model matrix shows honestly that a schema-valid but fabricated statement passes the hard layers and is only flagged. Its backends are a dependency-free heuristic and an optional LLM judge.
- The agent membrane mediates tool *effects*, not the model's prose; its shell check is a **structural allowlist** (a shell lexer resolving real program names) — obfuscation-resistant and fail-closed.
- **TCB:** the host runtime (`wasmtime`) and the operating system are assumed intact. WASI filesystem granularity is at the directory/permission level.
- This artifact does *not* use SMT/Z3 proof or hardware synthesis; those belong to the broader MetaSpace.Bio programme and are out of scope here. No figure above is a benchmark or a marketing number: every **[MEASURED]** item is a reproducible run.
- The structural shell policy, the dry-run learning mode, the threat-model matrix, the ratification cognitive brake and the full per-kind capability mediation were added in response to two rounds of external technical review; each ships with its own reproducible proof.

Reproducibility & availability

Source and proofs: `github.com/LemonScripter/metaspace-membrane`. One command reproduces the entire ledger: `python run_proofs.py`.

IP: patent pending. **MetaSpace.Bio Engine Project** ● Szőke László-Ferenc ● admin@metaspace.bio ● metaspace.bio